

Q. No.	Questions
UNIT 1	SECTION 1
1	ChargeGrid, an EV-charging network, ships an over-the-air (OTA) firmware update to its charging stations. The Testing phase validated the update only on the newest hardware revision (Rev-C) and skipped device-diversity testing across older field-deployed revisions (Rev-A, Rev-B), which still make up 60% of installed stations. Two weeks after rollout, Rev-A and Rev-B stations begin reporting intermittent charging failures. Analyze the specific consequence chain across the Testing, Deployment, and Maintenance phases that led to this outcome, and identify the exact testing gap responsible. (5 Marks)
2	A telemedicine platform's video-consultation feature passed 100% of its functional test cases in a controlled lab network, but field usage reveals frequent call drops and frozen video for patients in low-bandwidth rural areas a condition never simulated during Testing. Identify the specific SDLC gap responsible for this outcome and name the type of testing that should have been included to catch it before release. (5 Marks)
3	A team builds a low-code/no-code prototype of a customer-support chatbot's webhook logic purely to demo conversation flow to stakeholders. After approval, a developer copies the prototype's actual webhook integration code directly into the production chatbot "because it already worked," and the system fails under real user load within days of launch. Identify the development mistake and assess the risk introduced by reusing the throwaway prototype code in the production system. (5 Marks)
4	(i) MediNow, a telemedicine platform, is being built over 12 months under Waterfall, with the first-ever regulator review scheduled at Month 12 (Figure 1.2, left). A new mandatory consent-recording rule, issued by the state teleconsultation board in Month 3, was never incorporated because no review happened until Month 12 - by which point the platform is rejected outright. The Spiral alternative (Figure 1.2, right) instead schedules a regulator review at the end of every short risk-driven cycle, starting at Month 2. MediNow Telemedicine — Regulatory-Risk Detection: Spiral vs Waterfall Gap exposed: Waterfall's single end-of-project regulator checkpoint vs Spiral's per-cycle regulator review Fig- MediNow's Waterfall regulator-review gap vs Spiral's per-cycle regulator review Based on Figure, Compare the handling of the new consent-recording rule in the Waterfall and Spiral models. (3 Marks) (ii) MediNow, a telemedicine platform, is being built over 12 months under Waterfall, with the first-ever regulator review scheduled at Month 12 (Figure 1.2, left). A new mandatory consent-recording rule, issued

by the state teleconsultation board in Month 3, was never incorporated because no review happened until Month 12 - by which point the platform is rejected outright. The Spiral alternative (Figure 1.2, right) instead schedules a regulator review at the end of every short risk-driven cycle, starting at Month 2.

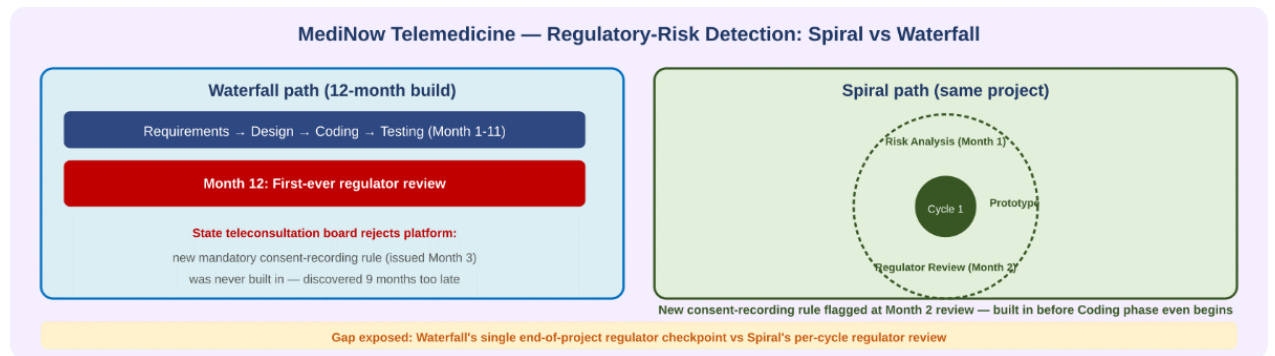


Fig. 1.2 - MediNow's Waterfall regulator-review gap vs Spiral's per-cycle regulator review

In the MediNow telemedicine project, identify one sub-component that is suitable for the Waterfall model and justify your choice.

(2 Marks)

UNIT 1

SECTION 2

- 5 A VP of Engineering scaling MLOps practices across 8 teams says: "SAFe, LeSS, and the Spotify Model are all just ways to run multiple Agile teams together, so picking one over another for our MLOps rollout is basically a coin flip."

Select a suitable structural aspect and compare the three frameworks for this MLOps rollout.

(5 Marks)

- 6 A healthcare organization retrain an AI-based patient risk prediction model after incorporating new clinical data. To shorten the release cycle, the ML team omits the bias and fairness validation step before deployment. During post-deployment monitoring, the model produces inconsistent predictions for different patient groups, requiring an emergency rollback.

Examine the impact of omitting the bias/fairness validation step on the ML retraining process using Lean principles.

(5 Marks)

- 7 VoltMeter's smart electricity-meter rollout across 3 cities below figure has a Gantt chart of 6 tasks: A-Firmware Certification (2 wks), B-Field Trial of 500 units (3 wks, after A), C-Regulatory Approval (2 wks, after B), D-Mass Production (4 wks, after C), F-City-Wide Rollout (3 wks, after D) - all marked critical - plus E-Utility-Grid Documentation (2.5 wks, running in parallel with B, not on the critical path).

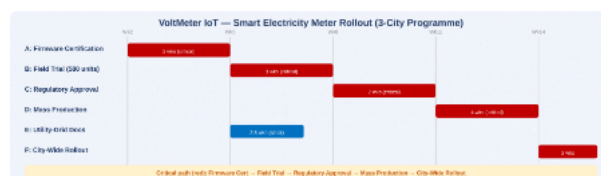


Fig- VoltMeter's Gantt chart with the critical path (red) and the parallel documentation task (blue)

Using above figure, calculate the total project duration and analyze the role of Task E in determining the project completion time.

(5 Marks)

- 8 (i) A last-mile autonomous delivery robot's SRS draft includes the following requirements:
(a) The system shall compute the shortest delivery route while avoiding pedestrian zones.

	(b) The system shall bring the robot to a complete stop within 200 milliseconds of detecting an obstacle. (c) The system shall encrypt all telemetry data in transit using TLS 1.3. Identify whether the above requirement as either a Functional Requirement (FR) or a Non-Functional Requirement (NFR). (3 Marks)
	(ii) A last-mile autonomous delivery robot's SRS draft includes the following requirements: (a) The system shall compute the shortest delivery route while avoiding pedestrian zones. (b) The system shall bring the robot to a complete stop within 200 milliseconds of detecting an obstacle. (c) The system shall encrypt all telemetry data in transit using TLS 1.3. Indicate the quality attribute for above Non-Functional Requirement (NFR) statements. (2 Marks)
UNIT 2	SECTION 1
9	A site reliability engineering (SRE) team is handling a critical production outage for a cloud application. They establish an emergency engineering loop to capture live telemetry and completely automate the delivery of a verified hotfix patch back into production. They document their resolution pipeline step-by-step: 1. Operate and Monitor Stage: The team analyzes live performance logs, server metrics, and automated alerts to find the root cause of the system crash. 2. Plan Stage: The gathered incident data is fed back to the project board to define technical requirements and create an urgent user story for the bug fix. 3. [Missing Stage A] 4. Build Stage: The newly written bug-fix code is automatically compiled and packaged into a standardized, deployable artifact to ensure clean environment parity. 5. Test Stage: The compiled patch runs through an automated evaluation suite to conduct unit, integration, and security checks to ensure no existing systems are broken. 6. [Missing Stage B] Identify the two missing DevOps lifecycle stages (Missing Stage A and Missing Stage B). State their exact formal names, and name any one standard software tool associated with each of these two missing stages. (5 Marks)
10	ExamPro Solutions, an ed-tech startup, runs its online examination portal as a set of containerized microservices deployed on a Kubernetes cluster. The operations team has observed the following behaviors: • Students access the application through a single, fixed URL, and their requests are internally routed to one of several backend containers without them ever knowing which specific container handled it. • Last week, one backend container crashed due to a memory issue during an ongoing exam, and within seconds Kubernetes automatically started a replacement container without any manual intervention from the operations team. • During a recent nationwide mock test, thousands of students logged in simultaneously, and their requests were spread evenly across all available containers so that no single container was overwhelmed. • On the day of a major entrance exam, traffic spiked sharply within minutes, and Kubernetes automatically launched additional container instances to handle the surge, scaling them back down once the exam window closed and traffic normalized. • The operations team also noted that this entire architecture built on containers rather than traditional virtual machines has made the portal easier to run reliably in production.

	Identify the Kubernetes capability demonstrated in each of the five highlighted behaviors described above, and justify why containers (in general) are considered suitable for production environments like this one. (5 Marks)
11	A development team is building an IoT application using Git. The project is currently stored only on a developer's computer. A second developer must work on a separate feature without disturbing the stable version, and the completed feature must later be integrated into the main project and shared through GitHub. Construct the appropriate Git workflow by selecting and arranging the required commands for the following stages: 1. Initialize the project as a Git repository. 2. Check the project state and stage the required files. 3. Save the changes as a commit. 4. Connect the local repository to a remote repository and push the main project. 5. Create and switch to a separate feature branch. 6. After completing the feature, return to the main branch and merge the feature branch. Construct the suitable Git commands for each stage and outline the purpose of the major steps. (5 Marks)
12	(i) A Docker-based application is running successfully in a production environment. During maintenance, the Docker daemon is restarted, but the running containers continue to operate without interruption. A company has deployed its web application using Docker containers in a production environment. The application handles thousands of user requests every minute and must remain available during routine maintenance. During a scheduled maintenance activity, the Docker daemon is restarted. However, the running containers continue to serve user requests without interruption. Predict the possibility of Docker's lower container-management layers ensure that running containers continue to operate even when the Docker daemon is restarted. (3 Marks) (ii) A Docker user enters a valid command through the Docker Client, but the requested container is not created or started. The container runtime itself is available on the system. Select the Docker architecture stage that should be investigated first and justify its priority. (2 Marks)
UNIT 2	SECTION 2
13	Nexoria Retail operates a monolithic e-commerce platform and is adopting DevOps to improve its software deployment process. The checkout module handles live customer payments and is a business-critical component where reliability is the highest priority. The organization is considering the following deployment strategies: Option A - Blue-Green Deployment: Two identical production environments enable instant rollback but require additional infrastructure. Option B - Rolling Deployment: Deploys updates gradually without extra infrastructure, but rollback is slower. Option C - Canary Deployment: Releases updates to a small percentage of users first, reducing risk but still exposing some customers. Organizational Constraints 1. No customer should experience deployment-related payment failures. 2. Additional infrastructure cost is acceptable for this critical module. Select the most suitable deployment strategy for the checkout module. Justify your choice that why the other two options fail to satisfy Nexoria's constraints, referencing cost, rollback speed, and customer exposure. (5 Marks)
14	A software company is developing an online railway ticket reservation system using Cleanroom Software

	Engineering. Different team members examine the system at different levels of abstraction during development. Some focus only on what the user sees, some study how the system's internal information changes, and others implement the detailed program logic. Identify the appropriate Box Model (Black Box, State Box, or Clear Box) represented by each statement. <table><tr><td>Case Studies</td></tr><tr><td>A. A tester verifies that entering valid travel details displays the correct ticket confirmation without examining the internal processing.</td></tr><tr><td>B. A system designer models how seat availability, booking status, and payment status change after each user action.</td></tr><tr><td>C. A programmer develops the algorithms and data structures that allocate seats and process ticket cancellations.</td></tr><tr><td>D. A customer checks whether cancelling a ticket correctly updates the refund amount displayed on the screen without knowing how it is calculated.</td></tr><tr><td>E. A software architect documents the transition of a booking from Available -> Reserved -> Confirmed -> Cancelled before coding begins.</td></tr></table> (5 Marks)	Case Studies	A. A tester verifies that entering valid travel details displays the correct ticket confirmation without examining the internal processing.	B. A system designer models how seat availability, booking status, and payment status change after each user action.	C. A programmer develops the algorithms and data structures that allocate seats and process ticket cancellations.	D. A customer checks whether cancelling a ticket correctly updates the refund amount displayed on the screen without knowing how it is calculated.	E. A software architect documents the transition of a booking from Available -> Reserved -> Confirmed -> Cancelled before coding begins.
Case Studies							
A. A tester verifies that entering valid travel details displays the correct ticket confirmation without examining the internal processing.							
B. A system designer models how seat availability, booking status, and payment status change after each user action.							
C. A programmer develops the algorithms and data structures that allocate seats and process ticket cancellations.							
D. A customer checks whether cancelling a ticket correctly updates the refund amount displayed on the screen without knowing how it is calculated.							
E. A software architect documents the transition of a booking from Available -> Reserved -> Confirmed -> Cancelled before coding begins.							
15	An online shopping application is built using several microservices. During a flash sale, customers report that the payment process is taking unusually long. The application includes the following request path: User Request → Web Service → Payment Service → Database → External Payment API The cloud infrastructure, containers, and individual services appear to be running, but the overall transaction is slow. By the given scenario, predict how an APM tool can investigate the given performance problem by outlining the information it should monitor and tracing the distributed transaction path to locate the actual source of the delay. (5 Marks)						
16	(i) A quality manager is performing a certified software reliability assessment on an autonomous vehicle braking module. The manager measures specific quantitative metrics during the statistical testing verification phase. Based on the core reliability parameters defined in Cleanroom metrics, identify the following scenarios which would indicate a successful certified reliability growth pattern for the module? (Select all that apply) • A) The failure intensity decreases significantly over a sustained unit of operational testing time. • B) The Mean Time To Failure (MTTF) systematically scales downward toward zero as testing progresses. • C) The defect rate per thousand lines of code (KLOC) steadily minimizes over time as errors are eliminated. • D) The module demonstrates a high Mean Time To Failure (MTTF), expressing long expected operational times before breaking. (3 Marks) (ii) The manager assumes that a successful compilation guarantees the software is fully reliable and error-free. (1 Mark) Identify the hidden assumption made by the project manager regarding successful code compilation. Outline why this assumption is incorrect and why statistical testing is still required before certifying software reliability. (2 Marks) (2 Marks)						
UNIT 3	SECTION 1						
17	PayFlow, a startup with 6 engineers, is developing a payment reconciliation platform that is expected to process 50,000 transactions per day within a year. Before development began, the team selected a Layered (N-Tier) Architecture (Figure) consisting of						

Presentation, Business Logic, Data Access, and Data Store layers. The Business Logic layer communicates with an external payment gateway. During the architecture review, the team focused only on selecting the architecture pattern and defining its components. They did not examine whether the payment gateway's 3-second response time under heavy load could affect system performance.

Three months after deployment, when the system handled 15,000 transactions per day, users experienced reconciliation delays of more than one minute during peak hours. The team later found that the payment gateway was the main cause of the performance problem.

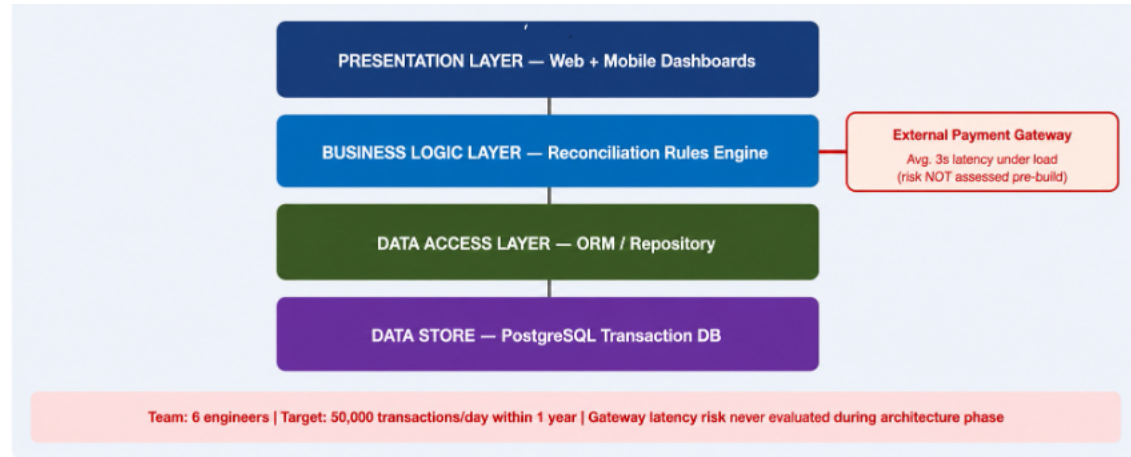


Figure: Payflow Reconciliation Platform: Layered Architecture

Determine whether the architecture planning was sufficient to reduce technical risks before development started, and support your answer by illustrating one architecture-phase activity that should have been carried out to reduce the risk.

(5 Marks)

- 18** RouteWorks, a logistics company, is developing an internal route-optimization system that will be maintained by its 6-member development team for at least 8 years.
- The team is considering two programming languages:
- Option 1 - C++: None of the team members have previous experience with C++, but it is expected to run the optimization program three times faster than Python.
 - Option 2 - Python: Five of the six developers have more than 4 years of Python experience, and libraries such as NumPy and Google OR-Tools are readily available for route optimization.
- The company also knows that developers usually write a similar number of lines of code each day, but higher-level languages allow more functionality to be developed in the same time. Python applications currently complete the overnight optimization process in 40 minutes, while the available batch-processing window is 6 hours, leaving sufficient time for future growth.
- Some managers believe that choosing Python may appear less advanced to the company's board.
- Compare the use of C++ and Python for this project by considering execution speed, developer productivity, and the available batch-processing time. Support your answer by choosing the language that is more suitable for RouteWorks' requirements.

(5 Marks)

- 19** Cobalt Systems is running two contracts at once. Contract Alpha is a fixed-price, fixed-scope government border-control system, legally required to have a complete, signed specification before implementation begins, with scope changes prohibited except through a formal contract amendment, and a hard 14-month delivery deadline. Contract Beta is a fast-evolving retail loyalty app for a startup client whose feature priorities shift roughly every two weeks based on live user data. Cobalt's engineering director proposes standardizing on a single construction approach for both contracts: Agile, with 2-week sprints and continuously evolving scope, arguing, 'it worked great for Beta, so we should use it everywhere for consistency.'

	Justify the suitability of standardizing the Agile construction approach for both contracts by comparing their requirements and identifying the consequence of applying Contract Beta's approach to Contract Alpha. (5 Marks)															
20	(i) NimbusCloud's QA lead must choose one major defect-detection practice for a 9-month software migration project. The project has a fixed team of three developers and must meet a strict security compliance deadline. The available information is as follows: • Code review (including pull request reviews) finds 60%-70% of defects before testing and costs much less than fixing defects during testing. • Formal code inspection (with defined team roles and review meetings) finds 60%-90% of defects but requires several developers to participate in scheduled review sessions. • Pair programming detects defects while coding but reduces development speed because two developers work on the same task. • Without these practices, most defects are found only during system testing, where each defect takes about 10 times more effort to fix than if it had been found during construction. Figure shows the defect detection rates of these practices. The team can adopt only one major practice-either formal code inspection or pair programming-in addition to its existing pull request reviews. Pre-Testing Defect-Catch Rate by Construction Practice <table border="1"><thead><tr><th>Practice</th><th>Defect Catch Rate</th><th>Additional Info</th></tr></thead><tbody><tr><td>Code Review (formal+PR)</td><td>60-70%</td><td></td></tr><tr><td>Formal Inspection</td><td>60-90%</td><td></td></tr><tr><td>Pair Programming (throughput halved)</td><td>Real-time</td><td></td></tr><tr><td>No Practice (10x fix cost)</td><td>Late only</td><td></td></tr></tbody></table> Compare formal code inspection and pair programming for NimbusCloud's project by considering the team size, project deadline, development speed, and defect detection rates. Select the more suitable practice with appropriate justification. (3 Marks) (ii) Based on your selection in Part (A), evaluate whether the team should continue using pull request reviews in addition to the chosen practice, considering the defect detection rates and the 10-times higher cost of fixing defects during system testing. (2 Marks)	Practice	Defect Catch Rate	Additional Info	Code Review (formal+PR)	60-70%		Formal Inspection	60-90%		Pair Programming (throughput halved)	Real-time		No Practice (10x fix cost)	Late only	
Practice	Defect Catch Rate	Additional Info														
Code Review (formal+PR)	60-70%															
Formal Inspection	60-90%															
Pair Programming (throughput halved)	Real-time															
No Practice (10x fix cost)	Late only															